

CS 601.471/671 NLP: Self-supervised Models

Homework 3: Building Your Neural Network!

For homework deadline and collaboration policy, check the calendar on the course website*

Name: _____

Collaborators, if any: _____

Sources used for your homework, if any: _____

This assignment is focusing on understanding the fundamental properties of neural networks and their training.

Homework goals: After completing this homework, you should be comfortable with:

- thinking about neural networks
- key implementation details of NNs, particularly in PyTorch,
- explaining and deriving Backpropagation,
- debugging your neural network in case it faces any failures.

How to hand in your written work: via Gradescope.

Collaboration: Make certain that you understand the course collaboration policy, described on the course website. You may discuss the homework to understand the problems and the mathematics behind the various learning algorithms, but you are **not allowed to share problem solutions with any other students. You must write the solutions individually.**

Typesetting: We strongly recommend typesetting your homework, especially if you have sloppy handwriting! :) We will provide a LaTeX template for homework solutions.

*<https://self-supervised.cs.jhu.edu/sp2025/>

1 Concepts, intuitions and big picture

1. Suppose you have built a neural network. You decide to initialize the weights and biases to be zero. Which of the following statements are True? (Check all that apply)
- ☐ Each neuron in the first hidden layer will perform the same computation. So even after multiple iterations of gradient descent each neuron will be computing the same thing as other neurons in the same layer.
 - ☐ Each neuron in the first hidden layer will perform the same computation in the first iteration. But after one iteration of gradient descent they will learn to compute different things because we have “broken symmetry”.
 - ☐ Each neuron in the first hidden layer will compute the same thing, but neurons in different layers will compute different things, thus we have accomplished “symmetry breaking” as described in lecture.
 - ☐ The first hidden layer’s neurons will perform different computations from each other even in the first iteration; their parameters will thus keep evolving in their own way.

Answer: The first option is correct.

2. Vectorization allows you to compute forward propagation in an L -layer neural network without an explicit for-loop (or any other explicit iterative loop) over the layers $l = 1, 2, \dots, L$. True/False?
- ☐ True
 - ☐ False

Answer: False; We cannot avoid the for-loop iteration over the computations among layers.

3. The $\tanh$ activation usually works better than sigmoid activation function for hidden units because the mean of its output is closer to zero, and so it centers the data better for the next layer. True/False?
- ☐ True
 - ☐ False

Answer: True, as seen in lecture the output of the tanh is between -1 and 1, it thus centers the data which makes the learning simpler for the next layer.

4. Which of the following techniques does NOT prevent a model from overfitting?
- ☐ Data augmentation
 - ☐ Dropout
 - ☐ Early stopping
 - ☐ None of the above

Answer: None of the above

5. Why should dropout be applied during training? Why should dropout NOT be applied during evaluation?

Answer: Training - reduce overfitting. Inference - dropout randomly disables neurons, which leads to random behavior during evaluation

6. Explain why initializing the parameters of a neural net with a constant is a bad idea.

Answer: Symmetry - mirrored input / output. Stagnation - same gradient during backprop.

7. You design a fully connected neural network architecture where all activations are sigmoids. You initialize the weights with large positive numbers. Is this a good idea? Explain your answer.

Answer: Large W causes Wx to be large. When Wx is large, the gradient is small for sigmoid activation function. Hence, we will encounter the vanishing gradient problem.

8. Explain what is the importance of “residual connections”.

Answer: Helps to address the vanishing gradient problem, and helps to prevent degradation in deep networks.

9. What is cached (“memoized”) in the implementation of forward propagation and backward propagation?
- ☐ Variables computed during forward propagation are cached and passed on to the corresponding backward propagation step to compute derivatives.
 - ☐ Caching is used to keep track of the hyperparameters that we are searching over, to speed up computation.
 - ☐ Caching is used to pass variables computed during backward propagation to the corresponding forward propagation step. It contains useful values for forward propagation to compute activations.

Answer: The “cache” records values from the forward propagation units and sends it to the backward propagation units because it is needed to compute the chain rule derivatives. So only the first one is correct.

10. Which of the following statements is true?

- ☐ The deeper layers of a neural network are typically computing more complex features of the input than the earlier layers.
- ☐ The earlier layers of a neural network are typically computing more complex features of the input than the deeper layers.

Answer: The first one is correct.

2 Revisiting Jacobians

Recall that Jacobians are generalizations of multi-variate derivatives and are extremely useful in denoting the gradient computations in computation graph and Backpropagation. A potentially confusing aspect of using Jacobians is their dimensions and so, here we're going to focus on understanding Jacobian dimensions.

Recap: Let's first recap the formal definition of Jacobian. Suppose $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a function that takes a point $\mathbf{x} \in \mathbb{R}^n$ as input and produces the vector $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^m$ as output. Then the Jacobian matrix of $\mathbf{f}$ is defined to be an $m \times n$ matrix, denoted by $\mathbf{J}_{\mathbf{f}}(\mathbf{x})$, whose (i, j) th entry is $\mathbf{J}_{ij} = \frac{\partial f_i}{\partial x_j}$, or:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^\top f_1 \\ \vdots \\ \nabla^\top f_m \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Examples: The shape of a Jacobian is an important notion to note. A Jacobian can be a vector, a matrix, or a tensor of arbitrary ranks. Consider the following special cases:

- If f is a scalar and $\mathbf{w}$ is a $d \times 1$ column vector, the Jacobian of f with respect to $\mathbf{w}$ is a row vector with $1 \times d$ dimensions.
- If $\mathbf{y}$ is a $n \times 1$ column vector and $\mathbf{z}$ is a $d \times 1$ column vector, the Jacobian of $\mathbf{z}$ with respect to $\mathbf{y}$, or $\mathbf{J}_{\mathbf{z}}(\mathbf{y})$ is a $d \times n$ matrix.
- Suppose $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{l \times p \times q}$. Then the Jacobian $\mathbf{J}_{\mathbf{A}}(\mathbf{B})$ is a tensor of shape $(m \times n) \times (l \times p \times q)$. More broadly, the shape of the Jacobian is determined as (shape of the output) $\times$ (shape of the input).

Problem setup: Suppose we have:

- X , an $n \times d$ matrix, $x_i \in \mathbb{R}^{d \times 1}$ correspond to the rows of $X = [x_1, \dots, x_n]^\top$
- Y , a $n \times k$ matrix
- W , a $k \times d$ matrix and w , a $d \times 1$ vector

For the following items, compute (1) the shape of each Jacobian, and (2) an expression for each Jacobian:

1. $f(w) = c$ (constant)
Answer: Dimension: $1 \times d$, Jacobian: $\mathbf{J}_f(w) = [0, \dots, 0]$
2. $f(w) = \|w\|^2$ (squared L2-norm)
Answer: Dimension: $1 \times d$, Jacobian: $\mathbf{J}_f(w) = 2w^\top$
3. $f(w) = w^\top x_i$ (vector dot product)
Answer: Dimension: $1 \times d$, Jacobian: $\mathbf{J}_f(w) = x_i^\top$
4. $f(w) = Xw$ (matrix-vector product)
Answer: Dimension: $n \times d$, Jacobian: $\mathbf{J}_f(w) = [x_1, \dots, x_n]^\top = X$
5. $f(w) = w$ (vector identity function)
Answer: Dimension: $d \times d$, Jacobian: $\mathbf{J}_f(w) = I_d$
6. $f(w) = w^2$ (element-wise power)
Answer: Dimension: $d \times d$, Jacobian: $\mathbf{J}_f(w) = 2\text{diag}(w)$
7. **Extra Credit:** $f(W) = XW^\top$ (matrix multiplication)

Answer: Dimension: $n \times k \times k \times d$, Jacobian: $\mathbf{J}_f(W)_{a,b,c,d} = \begin{cases} 0 & \text{if } b \neq c \\ X_{ad} & \text{if } b = c \end{cases}$

3 Activations Per Layer, Keeps Linearity Away!

Based on the content we saw at the class lectures, answer the following:

1. Why are activation functions used in neural networks? *Answer: Without activation functions, neural networks could only learn linear functions since they can be collapsed into a single matrix of parameters. Activation functions are used to introduce non-linearities in the network to learn nonlinear functions.*
2. Write down the formula for three common action functions (sigmoid, ReLU, Tanh) and their derivatives (assume scalar input/output). Plot these activation functions and their derivatives on $(-\infty, +\infty)$. *Answer:*

$$S(x) = \frac{1}{1 + e^{-x}}$$

$$S'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}}\right) = S(x)(1 - S(x))$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

$$\tanh'(x) = 1 - \tanh^2(x)$$

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

Their visualizations can be found [here](#).

3. What is the “vanishing gradient” problem? (respond in no more than 3 sentences) Which activation functions are subject to this issue and why? (respond in no more than 3 sentences).
Answer: As partial derivatives approach zero, updates to model weights become increasingly small. This problem has a cascading effect, since gradients from later layers are usually relevant for the computation of gradients in earlier layers. This can lead to a situation where the model stops “learning.” This happens for sigmoid and tanh functions for high (absolute) input values. This is why the ReLU function was introduced.
4. Why zero-centered activation functions impact the results of Backprop?
Answer: Example Explanation
5. Remember the Softmax function $\sigma(\mathbf{z})$ and how it extends sigmoid to multiple dimensions? Let’s compute the derivative of Softmax for each dimension. Prove that:

$$\frac{d\sigma(\mathbf{z})_i}{dz_j} = \sigma(\mathbf{z})_i(\delta_{ij} - \sigma(\mathbf{z})_j)$$

where δ_{ij} is the Kronecker delta function.* *Answer: If $i \neq j$ then*

$$\frac{d\sigma(\mathbf{z})_i}{dz_j} = \frac{d}{dz_j} \left(\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \right) = \frac{-e^{z_i} e^{z_j}}{(\sum_{j=1}^K e^{z_j})^2} = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \cdot \frac{-e^{z_j}}{\sum_{j=1}^K e^{z_j}} = \sigma(\mathbf{z})_i(-\sigma(\mathbf{z})_j) = \sigma(\mathbf{z})_i(\delta_{ij} - \sigma(\mathbf{z})_j)$$

otherwise $i = j$ and

$$\frac{d\sigma(\mathbf{z})_i}{dz_j} = \frac{d}{dz_j} \left(\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \right) = \frac{e^{z_i} \sum_{j=1}^K e^{z_j} - e^{z_i} e^{z_j}}{(\sum_{j=1}^K e^{z_j})^2} = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \cdot \frac{\sum_{j=1}^K e^{z_j} - e^{z_j}}{\sum_{j=1}^K e^{z_j}} = \sigma(\mathbf{z})_i(\delta_{ij} - \sigma(\mathbf{z})_j)$$

6. Use the above point to prove that the Jacobian of the Softmax function is the following:

$$\mathbf{J}_\sigma(\mathbf{z}) = \text{diag}(\sigma(\mathbf{z})) - \sigma(\mathbf{z})\sigma(\mathbf{z})^\top$$

*https://en.wikipedia.org/wiki/Kronecker_delta

where $\text{diag}(\cdot)$ turns a vector into a diagonal matrix. Also, note that $\mathbf{J}_\sigma(\mathbf{z}) \in \mathbb{R}^{K \times K}$.

Answer:

$$\mathbf{J}_\sigma(\mathbf{z})_{ij} = \frac{\partial \sigma(\mathbf{z})_i}{\partial z_j} = \sigma(\mathbf{z})_i(\delta_{ij} - \sigma(\mathbf{z})_j) = \sigma(\mathbf{z})_i\delta_{ij} - \sigma(\mathbf{z})_i\sigma(\mathbf{z})_j$$

$$\text{since } \sigma(\mathbf{z})_i\delta_{ij} = \text{diag}(\sigma(\mathbf{z}))_{ij}, \text{ and } \sigma(\mathbf{z})_i\sigma(\mathbf{z})_j = (\sigma(\mathbf{z})^\top \sigma(\mathbf{z}))_{ij}$$

$$\mathbf{J}_\sigma(\mathbf{z})_{ij} = \text{diag}(\sigma(\mathbf{z}))_{ij} - (\sigma(\mathbf{z})^\top \sigma(\mathbf{z}))_{ij}$$

$$\mathbf{J}_\sigma(\mathbf{z}) = \text{diag}(\sigma(\mathbf{z})) - \sigma(\mathbf{z})^\top \sigma(\mathbf{z})$$

Note: here we treat $\sigma(\mathbf{z})$ as column vector

4 Simulating XOR

1. Can a single-layer network simulate (represent) an XOR function on $\mathbf{x} = [x_1, x_2]$?

$$y = \text{XOR}(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{x} = (0, 1) \text{ or } \mathbf{x} = (1, 0) \\ 0, & \text{if } \mathbf{x} = (1, 1) \text{ or } \mathbf{x} = (0, 0). \end{cases}$$

Explain your reasoning using the following single-layer network definition: $\hat{y} = \text{ReLU}(W \cdot \mathbf{x} + b)$

Answer: *The XOR problem is not linearly separable. Therefore a single perceptron can not converge to a solution. We must use a multi-layer perceptron (multi-layer feedforward net) to solve this problem.*

2. Repeat (1) with a two-layer network:

$$\begin{aligned} \mathbf{h} &= \text{ReLU}(W_1 \cdot \mathbf{x} + \mathbf{b}_1) \\ \hat{y} &= W_2 \cdot \mathbf{h} + b_2 \end{aligned}$$

Note that this model has an additional layer compared to the earlier question: an input layer $\mathbf{x} \in \mathbb{R}^2$, a hidden layer $\mathbf{h}$ with ReLU activation functions that are applied component-wise, and a linear output layer, resulting in scalar prediction $\hat{y}$. Provide a set of weights W_1 and W_2 and biases $\mathbf{b}_1$ and b_2 such that this model can accurately model the XOR problem.

Answer: *Example Explanation*

3. Consider the same network as above (with ReLU activations for the hidden layer), with an arbitrary differentiable loss function $\ell : \{0, 1\} \times \{0, 1\} \rightarrow \mathbb{R}$ which takes as input $\hat{y}$ and y , our prediction and ground truth labels, respectively. Suppose all weights and biases are initialized to zero. Show that a model trained using standard gradient descent will not learn the XOR function given this initialization.

Answer: *Let's consider the following network:*

$$\begin{aligned} \mathbf{h} &= \text{ReLU}(W_1 \cdot \mathbf{x} + \mathbf{b}_1) \\ \hat{y} &= W_2 \cdot \mathbf{h} + b_2 \end{aligned}$$

Initially for any $\mathbf{x}$ we have $\mathbf{h} = 0$. We can then see that $\frac{\partial \ell}{\partial W_2} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_2} = \frac{\partial \ell}{\partial \hat{y}} \mathbf{h} = 0$. Thus, weights W_2 will not be updated on our first pass when using standard gradient descent optimization techniques. Similarly, one can show that $\frac{\partial \ell}{\partial W_1} = 0$, $\frac{\partial \ell}{\partial b_2} = 1$ and $\frac{\partial \ell}{\partial b_1} = 0$. This means all these parameters except for b_2 will not change and will remain zero after a single round of gradient descent. Thus the output of the neural network will always equal to b_2 regardless of the input, which can not simulate XOR.

4. **Extra Credit:** Now let's consider a more general case than the previous question: we have the same network with an arbitrary hidden layer activation function:

$$\begin{aligned} \mathbf{h} &= f(W_1 \cdot \mathbf{x} + \mathbf{b}_1) \\ \hat{y} &= W_2 \cdot \mathbf{h} + b_2 \end{aligned}$$

Show that if the initial weights are any uniform constant, then gradient descent will not learn the XOR function from this initialization.

Answer: It would be simpler to analyze in the following formulation:

$$\begin{aligned}h_1 &= f(w_{11} \cdot \mathbf{x} + b_{11}) \\h_2 &= f(w_{12} \cdot \mathbf{x} + b_{12}) \\\hat{y} &= w_{21}h_1 + w_{22}h_2 + b_2\end{aligned}$$

From the question statement, we know that $w_{11} = w_{12}$, $b_{11} = b_{12}$, and $w_{21} = w_{22}$. From this equality, it must be the case that $\frac{\partial \ell}{\partial w_{21}} = \frac{\partial \ell}{\partial w_{22}}$. Thus, the two weights w_{21} and w_{22} will receive the same update and will remain equal. One can show a similar equality for all the parameters:

$$\begin{aligned}\frac{\partial \ell}{\partial w_{11}} &= \frac{\partial \ell}{\partial w_{12}} \\\frac{\partial \ell}{\partial b_{11}} &= \frac{\partial \ell}{\partial b_{12}}\end{aligned}$$

As a result, we will always have $h_1 = h_2$. These equalities imply the network is equivalent to a perceptron with a single node on its hidden layer, so it is equivalent to a single layer perceptron. As stated in question 1, a single layer perceptron can not solve the XOR problem.

5 Neural Nets and Backpropagation

Draw the computation graph for $f(x, y, z) = \ln x + \exp(y) \cdot z$. Each node in the graph should correspond to only one simple operation (addition, multiplication, exponentiation, etc.). Then we will follow the forward and backward propagation described in class to estimate the value of f and partial derivatives $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$ at $[x, y, z] = [1, 3, 2]$. For each step, show your work.

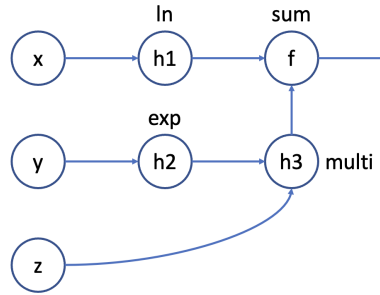
A computation graph, so elegantly designed
With nodes and edges, so easily combined
It starts with inputs, a simple array
And ends with outputs, in a computationally fair way

Each node performs, an operation with care
And passes its results, to those waiting to share
The edges connect, each node with its peers
And flow of information, they smoothly steer

It's used to calculate, complex models so grand
And trains neural networks, with ease at hand
Backpropagation, it enables with grace
Making deep learning, a beautiful race
-ChatGPT Feb 3 2023

1. Draw the computation graph for $f(x, y, z) = \ln x + \exp(y) \cdot z$. The graph should have three input nodes for x, y, z and one output node f . Label each intermediate node h_i .

Answer:



2. Run the forward propagation and evaluate f and h_i ($i = 1, 2, \dots$) at $[x, y, z] = [1, 3, 2]$.

Answer:

$$h_1 = \ln 1 = 0, \quad h_2 = e^3, \quad h_3 = 2 \cdot e^3, \quad f = h_1 + h_3 = 2 \cdot e^3$$

3. Run the backward propagation and give partial derivatives for each intermediate operation, i.e., $\frac{\partial h_i}{\partial x}, \frac{\partial h_j}{\partial y}$, and $\frac{\partial f}{\partial h_i}$. Evaluate the partial derivatives at $[x, y, z] = [1, 3, 2]$.

Answer:

$$\frac{\partial f}{\partial h_1} = 1, \quad \frac{\partial f}{\partial h_3} = 1, \quad \frac{\partial h_1}{\partial x} = \frac{1}{x} = 1$$

$$\frac{\partial h_3}{\partial h_2} = z = 2, \quad \frac{\partial h_3}{\partial z} = h_2 = e^3, \quad \frac{\partial h_2}{\partial y} = \exp(y) = e^3$$

4. Aggregate the results in (c) and evaluate the partial derivatives $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$ with chain rule. Show your work.

Answer:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial h_1} \frac{\partial h_1}{\partial x} = 1, \quad \frac{\partial f}{\partial y} = \frac{\partial f}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial y} = 2 \cdot e^3, \quad \frac{\partial f}{\partial z} = \frac{\partial f}{\partial h_3} \frac{\partial h_3}{\partial z} = e^3$$

6 Programming

In this programming homework, we will

- implement MLP-based classifiers for the sentiment classification task of homework 1.

Skeleton Code and Structure: The code base for this homework can be found at [this GitHub repo](#) under the hw3 directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `mlp.py` reuse the sentiment classifier on movie reviews you implemented in homework 1, with additional requirements to implement MLP-based classifier architectures and forward pass .
- `main.py` provides the entry point to run your implementations `mlp.py`
- `hw3.md` provides instructions on how to setup the environment and run each part of the homework in `main.py`

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a `# TODO` at the corresponding blank in the code.

TODOs (Copy from your HW1). We are reusing most of the `model.py` from homework 1 as the starting point for the `mlp.py` - you will see in the skeleton that they look very similar. Moreover, in order to make the skeleton complete, for all the `# TODO (Copy from your HW1)`, please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in homework 1.)

Submission: Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a .zip file

6.1 MLP-based Sentiment Classifier

In both homework 1 & 2, our implementation of the `SentimentClassifier` is essentially a single-layer feedforward neural network that maps input features directly to 2-dimensional output logits. In this part of the programming homework, we will expand the architecture of our classifier to multi-layer perceptron (MLP).

6.1.1 Reuse Your HW1 Implementation

TODOs (Copy from your HW1): for all the `# TODO (Copy from your HW1)` in `mlp.py`, please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in the `model.py` in homework 1).

6.1.2 Build MLPs

Remember from the lecture that MLP is a multi-layer feedforward network with perceptrons as its nodes. A perceptron consists of non-linear activation of the affine (linear) transformation of inputs.

TODOs: Complete the `__init__` and forward function of the `SentimentClassifier` class in `mlp.py` to build MLP classifiers that supports custom specification of architecture (i.e. number and dimension of hidden layers)

Hint: check the comments in the code for specific requirements about input, output, and implementation. Also, check out the document of [nn.ModuleList](#) about how to define and implement forward pass of MLPs as a stack of layers.

6.1.3 Train and Evaluate MLPs

We provide in `main.py` several MLP configurations and corresponding recipes for training them.

TODOs Once you finished [subsubsection 6.1.2](#), you can run `load_data_mlp` and `explore_mlp_structures` to train and evaluate these MLPs and paste two sets of plots here:

- 4 plots of train & dev loss for each MLP configuration
- 2 plots of dev losses and accuracies across MLP configurations

and describe in 2-3 sentences your findings.

Hint: what are the trends of train & dev loss and are they consistent across different configurations? Is deeper models always better? Why?

your plots and answer:

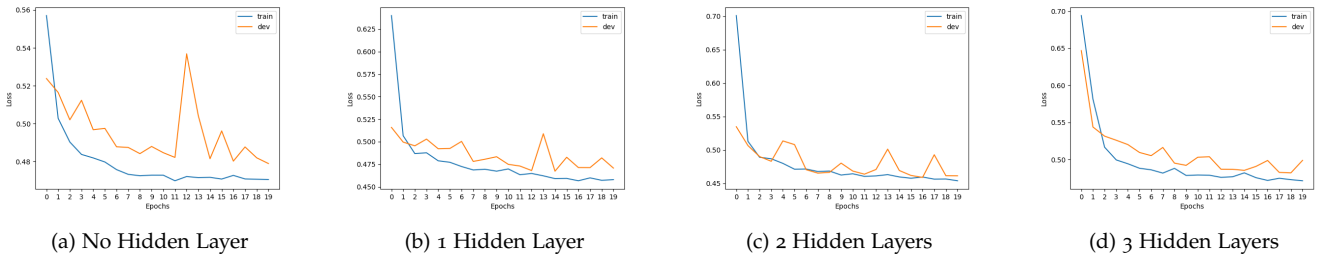


Figure 1: loss of different MLP architectures

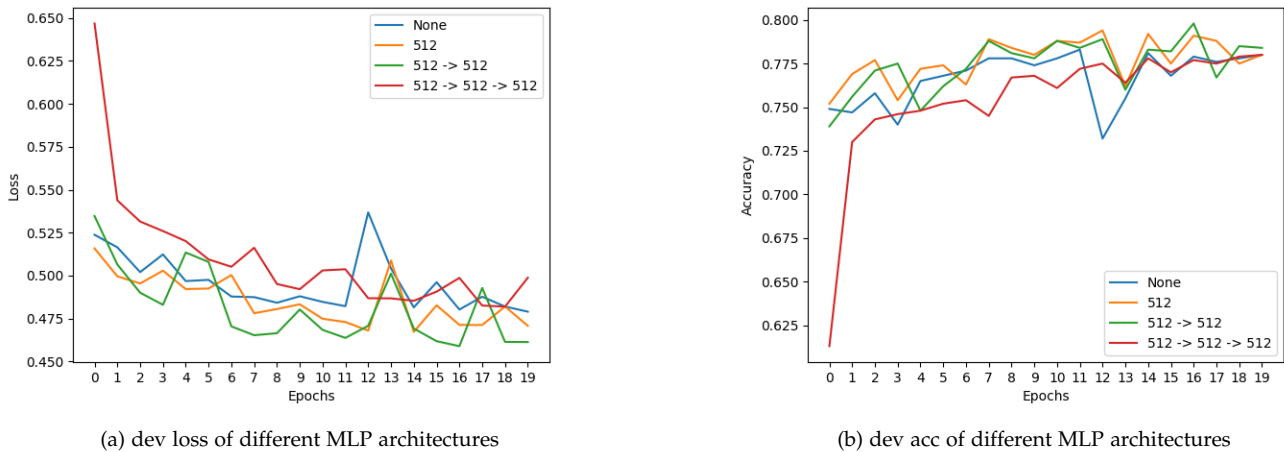


Figure 2: loss and acc of different MLP architectures

Answer: Training losses decrease relatively steadily across configurations while dev losses fluctuate a lot and are often higher than training losses. While with 1 or 2 hidden layers the MLP performs better, with 3 hidden layers it performs worse than the base shallow model. Some possible reasons are that as the network goes deeper, it's harder to optimize which requires dedicated tuning of hyper-parameters, or it's because the model is too over-parameterized at hidden layer = 3 for this dataset.

6.1.4 Embrace Non-linearity: The Activation Functions

Remember we have learned why adding non-linearity is useful in neural nets and gotten familiar with several non-linear activation functions both in the class and [section 3](#). Now it is time to try them out in our MLPs!

Note: for the following TODO and the TODO in [subsubsection 6.1.5](#), we fix the MLP structure to be with a single 512-dimension hidden layer, as specified in the code. You only need to run experiments on this architecture.

TODOs: Read and complete the missing lines of the two following functions:

- `__init__` function of the `SentimentClassifier` class: define different activation functions given the input activation type.

Hint: we have provided you with a demonstration of defining the Sigmoid activation, you can search for the other `nn.<activation>` in PyTorch documentation.

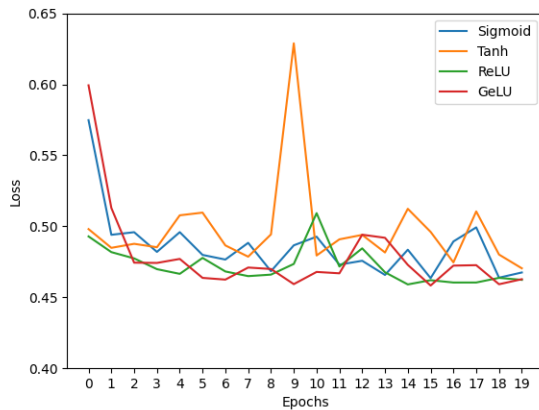
- `explore_mlp_activations` in `main.py`: iterate over the activation options, define the corresponding training configurations, train and evaluate the model, and visualize the results. Note: you only need to generate the

plots of dev loss and dev acc across different configurations, by calling `visualize_configs`, you **do not** need to plot the train-dev loss curves for each configuration (i.e. no need to call `visualize_epochs`). We provide you with a few choices of common activation functions, but feel free to try out the others.

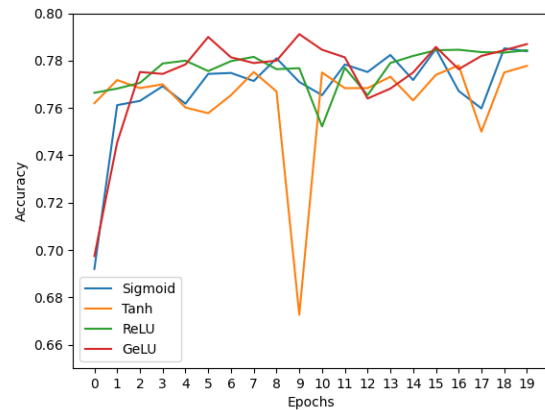
Hint: You can refer to `explore_mlp_structure` as a demonstration of how to define training configurations with fixed hyper-parameters & iterate over hyper-parameters/design choices of interests (e.g. hidden dimensions, choice of activation), and plot the evaluation results across configurations.

Once you complete the above functions, run `explore_mlp_activations` and paste the two generated plots here. Describe in 2-3 sentences your findings.

your plots and answer:



(a) dev loss of different MLP architectures



(b) dev acc of different MLP architectures

Figure 3: loss and acc of different activations

Answer: While with ReLU the accuracies fluctuate the most, there seem to be no significant performance differences between different activations. This might be due to the simplicity of the dataset and model structure, makes it hard to distinguish the effects of activations.

6.1.5 Hyper-parameter Tuning: Learning Rate

The training process mostly involves learning model parameters, which are automatically performed by gradient-based methods. However, certain parameters are “unlearnable” through gradient optimization while playing a crucial role in affecting model performance, for example, learning rate and batch size. We typically refer to these parameters as *Hyper-parameters*.

We will now take the first step to tune these hyper-parameters by exploring the choices of one of the most important one - learning rate, on our MLP. (There are lots of tutorials on how to tune the learning rate manually or automatically in practice, for example [this note](#) can serve as a starting point.)

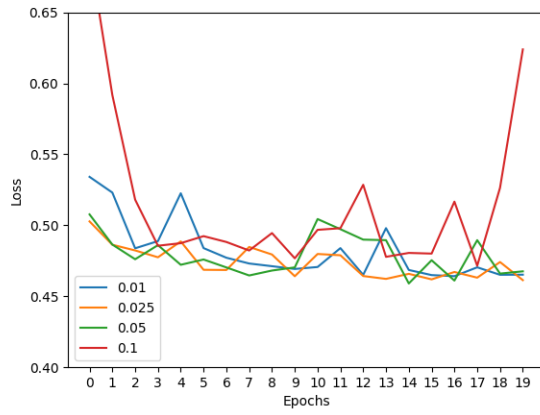
TODOs: Read and complete the missing lines in `explore_mlp_learning_rates` in `main.py` to iterate over different learning rate values, define the training configurations, train and evaluate the model, and visualize the results. Note: same as above, you only need to generate the plots of dev loss and dev acc across different configurations, by calling `visualize_configs`, you **do not** need to plot the train-dev loss curves for each configuration (i.e. no need to call `visualize_epochs`). We provide you with the default learning rate we set to start with, and we encourage you to add more learning rate values to explore and include in your final plots curves of **at least 4 different representative learning rates**.

Hint: again, you can checkout `explore_mlp_structure` as a demonstration for how to perform hyper-parameter search.

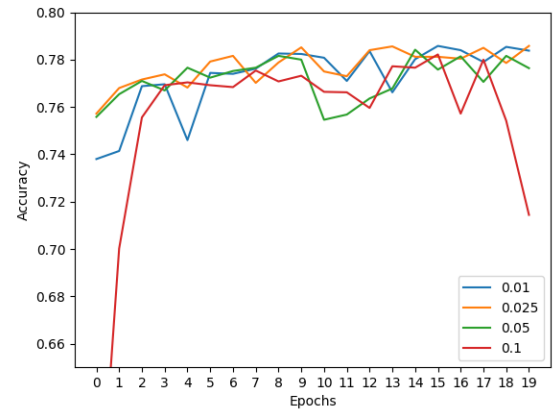
Once you complete the above functions, run `explore_mlp_learning_rates` and paste the two generated plots here. Describe in 2-3 sentences your findings.

your plots and answer:

Answer: While training with most of the learning rates behave similarly, at learning rate = 0.1 the loss becomes very unstable at the beginning and the end. This indicates that the optimization becomes harder when the learning rate is too large.



(a) dev loss of different MLP architectures



(b) dev acc of different MLP architectures

Figure 4: loss and acc of different learning rates

References